Validation and Error Patterns

Member 5 · Cybersecurity and DevOps

Version 1.0 / 30 October 2026

Audience: Members 1–4 implementing accounts, rooms/events, browser storage and ASR/AI. Source baseline: SRS v1.0 pages 3–11 and TEAM v1.0 pages 2–6, dated 25 October 2026.

This guide distinguishes shipped common code from integration requirements. The prototype has no complete authentication, room state, transcript cache, ASR adapter or AI pipeline. Helpers and fixtures are not proof those features exist. report.md and acceptance.md record actual outcomes.

1. The receiving boundary is the authority

Treat body, URL params, queries, headers, cookies, WebSocket frames, localStorage, webhook payloads, ASR/model output and retrieved pages as untrusted. TypeScript types and frontend checks alone do not validate runtime values.

Apply this order at every new route: bounded transport and Origin policy; verified authentication; strict schema; resource membership and state; domain operation/transaction; validated safe response. Public registration/login are explicit exceptions to authentication, never exceptions to validation or abuse limits. Resource lookup failures must not disclose another account's data.

• Reject unknown mutation fields instead of silently accepting mass assignment.

• Use type, length, range, enum and format checks; do not coerce mutation strings into booleans/numbers.

• Normalize email/display labels only by declared product semantics. Never trim, normalize, truncate or log passwords.

• Validation does not replace authorization, parameterized SQL, safe rendering, CSRF protection or transaction constraints.

• Do not make browser room names, account IDs, cached transcripts or model decisions authoritative.

Existing shared modules are under backend/src/security. app.ts installs the current HTTP boundary. Members 1–4 must mount helpers and add schemas at their own feature boundary.

2. Use strict schemas and typed parsed values

The delivered registrationSchema preserves password text, normalizes the declared email form and limits display names. createRoomSchema permits numeric durationSeconds from 120 to 1800 (default 600), title 5–200 and description up to 2000 characters. These are proposed SRS defaults; routes are not implemented yet.

```
import { z } from "zod";
import { validateBody, parseInput, createRoomSchema }
  from "./security/validation.js";

// Member 2 supplies requireAuth and createRoomHandler.
router.post("/rooms", requireAuth,
  validateBody(createRoomSchema), createRoomHandler);

const params = parseInput(
  z.strictObject({ id: z.uuid() }), req.params);
```

Express 5 query/params should not be reassigned. Use the returned value from parseInput; validate queries with historyQuerySchema (page ≥1, limit 1–50) and explicitly bound database result counts. Body parsing middleware replaces req.body only after successful validation.

Current media-token input is a legacy prototype DTO: roomName and identity, each 1–64 ASCII letters/digits/underscore/hyphen, no extra fields. The production authorizer must ignore these browser choices and derive the authorized room and account identity from verified server state. Member 2 should migrate to POST /sessions/:id/media-token with the shared error contract when their session API is ready.

Keep output DTOs explicit: omit password/refresh hashes, invitation secrets, provider credentials, prompts and transcripts from history/results. Do not serialize ORM objects wholesale.

3. Stable error contract and recovery

All current API failures return the same shape. X-Request-ID equals the response requestId; the server generates it and ignores attacker-supplied IDs.

```
{
  "error": {
    "code": "INVALID_INPUT",
    "message": "Check the request fields and try again."
  },
  "requestId": "server-generated-identifier"
}
```

Status	Stable code	Client recovery
400	INVALID_INPUT / INVALID_JSON	Correct fields or request encoding; no blind retry
401	UNAUTHENTICATED	Renew once through protected refresh or sign in
403	FORBIDDEN	Stop the action; do not reveal other users' resources
403	ORIGIN_NOT_ALLOWED / CSRF_INVALID	Use allowed app / reload a valid session
404	NOT_FOUND	Show unavailable resource without confirming ownership
409	STATE_CONFLICT	Fetch authoritative state before deciding next action
413	PAYLOAD_TOO_LARGE	Reduce payload; do not split speech to evade limits
415	UNSUPPORTED_MEDIA_TYPE	JSON only; no compressed request bodies
429	RATE_LIMITED	Honor Retry-After seconds; do not retry in a loop
503	SERVICE_UNAVAILABLE	Show degraded/unavailable state; bounded retry only
503	MEDIA_AUTHORIZATION_UNAVAILABLE	Auth/membership integration missing; not user denial
500	INTERNAL_ERROR	Generic failure and support request ID; no internals

Use AppError for known domain failures and allow unexpected exceptions to reach errorHandler. Never send raw Zod errors, exception.message, SQL errors, provider responses, stack traces, password values or user input back to the browser. Add new stable codes centrally with safe messages and negative tests.

```
import { AppError } from "./security/errors.js";
if (!member) throw new AppError(403, "FORBIDDEN");
if (session.state !== "active") {
  throw new AppError(409, "STATE_CONFLICT");
}
```

The frontend reads data.error.message as plain React text and should use code for recovery decisions. Do not branch on translated message strings. A service outage or incomplete capture must remain unavailable, never fabricated text, citations or scores.

4. Authentication, cookies, CSRF and media

Member 1 owns real authentication. Verify JWT signature with an explicit algorithm allowlist plus issuer, audience, expiry and relevant account/session revocation. Proposed access lifetime is 15 minutes. Keep access tokens in browser memory; no tokens in localStorage, URLs or logs. Refresh tokens are high-entropy opaque values stored only as hashes server-side, rotated atomically; replay revokes their family. Proposed refresh lifetime is seven days.

Refresh cookies must be Secure, HttpOnly, SameSite and deliberately scoped (for example /auth when refresh/logout live there). Scope names/path/domain consistently when clearing a stale cookie. Cookie transport alone does not implement CSRF defense.

```
import { csrfProtection } from "./security/http.js";
// bindRefreshSession must verify and load the refresh session.
router.post("/auth/refresh", bindRefreshSession,
  csrfProtection({
    allowedOrigins: config.allowedOrigins,
    expectedToken: locals => locals.refreshSession?.csrfToken,
  }), refreshHandler);
```

The snippet is integration pseudocode: Member 1 defines a typed refreshSession and a session-bound synchronizer token with at least 32 characters of cryptographic randomness. The helper rejects missing/foreign Origin and missing/wrong tokens using constant-time comparison. Never derive expectedToken from the same untrusted header/cookie pair; SameSite and CORS alone are insufficient.

Members 1/2 supply createApp({ authorizeMedia }). This hook must verify account identity, room/session membership, consent and active state before returning trusted identity/roomName. Missing hook fails closed with 503. Unauthorized users must receive 401/403 or non-disclosing 404 as appropriate. The optional local demo flag is prohibited in production and supplies none of these checks.

Existing grants expire within five minutes, name one room, permit only microphone publication and subscription, and disable data/admin/record/metadata-update privileges. Member 2 must revoke active permissions/remove participants on finish; token expiry does not terminate an existing connection. Enforce two human slots in a database transaction, separately from the transcription service identity.

5. Events, browser cache and safe display

Member 2: authenticate WebSocket upgrade and validate exact Origin before accepting it; enforce session membership on every subscription and operation. Limit frames to 16 KiB plus per-connection/per-account message rates. parseEvent checks size before JSON.parse and then applies a supplied strict schema. It is a helper, not an implemented WS server.

Event envelope must contain event_id, session_id, type, sequence and server_time plus validated payload (INT-02). Require valid UUIDs, supported enum types, nonnegative bounded sequences, ISO timestamps and per-type payload schemas. Ignore duplicates; reconcile gaps/reconnect against server state. Authenticate and verify any webhooks before processing and deduplicate by stable event ID.

Member 3: parse localStorage as unknown, catch JSON/storage/quota errors, validate schema/version/account/session IDs/expiry and server active state before rendering. Cache at most 1 MiB/session, namespaced per account; discard forged, stale or cross-account content. End/cancel/logout/account switch stops events and clears UI/cache in all tabs. Expiry is at most session end plus five minutes; remove stale cache at startup before rendering. A closed/offline browser cannot promise immediate physical deletion.

Render user/ASR/model text as React text, never dangerouslySetInnerHTML. Do not delete quotes/angle brackets and call it SQL/XSS protection. Use parameterized SQL and context-appropriate rendering. Treat cache text as a convenience only: it may never reconstruct official AI input after a server restart.

6. ASR, AI and retrieval trust boundaries

Member 4 must bind speaker identity to authenticated LiveKit tracks, not ASR/browser labels. Validate segment IDs, owner IDs, revisions, timestamps, finality and text length (≤4000 characters). Deduplicate revisions and cap canonical transcript memory at 1 MiB. Capture gaps/limits must mark evaluation unavailable. Freeze after at most five seconds of drain on finish, then purge after success/final failure and no later than five minutes after finish.

Validate AI output against known participant IDs, four finite scores each 0–25, bounded feedback, allowed verdict enums and at most five checked claims. Recompute totals in code. Require complete capture and at least 50 recognized words per participant before scoring; treat these values as the proposed baseline. Reject invented IDs, out-of-range/NaN values, unknown fields and inconsistent evidence. Do not persist prompt/full transcript/chronological quotes as feedback.

Treat speech, web pages and model output as data. Delimit evidence from instructions; give the model no DB write capability or unrestricted tools. A prompt containing "ignore prior instructions" is test input, never a permission change. The model cannot decide authorization.

Only the approved evidence adapter may retrieve material. Prefer provider-returned evidence IDs and accept citations only from those actual results. HTTPS source URLs are display metadata, not permission to fetch them. If server URL fetching is introduced, enforce an exact hostname allowlist, reject credentials/custom ports and all loopback/private/link-local IPs (including IPv6/mapped/encoded forms), validate all DNS answers, prevent DNS rebinding by binding the validated connection, recheck every redirect, and bound bytes/time. Network egress policy must reinforce this. A string-only URL check does not prevent SSRF.

No SSRF fetcher or AI adapter is delivered in this change. These are Member 4 implementation/review gates. Evaluation calls have a proposed 60-second timeout, at most one bounded transient retry and the five-minute transcript retention cap. Persist one final result per session with a uniqueness constraint; a lost buffer after restart means unavailable, not browser reconstruction.

7. Abuse, logging and operational integration

The current HTTP boundary accepts uncompressed application/json up to 16 KiB; nginx matches the body limit and returns a structured 413. Define an explicitly bounded endpoint-specific limit if a future legitimate payload needs more; never raise the global limit without review.

The single-instance IP limiter defaults to 60 requests per 15 minutes, exempts health and returns Retry-After. Storage is bounded to 10000 live keys; saturation rejects new keys rather than evicting limits. Spoofed X-Forwarded-For is ignored unless the request came from a configured individual proxy address. Nginx overwrites forwarding headers. Deployments with multiple API replicas need shared atomic counters.

Member 1 adds five failed logins/account/15 minutes with backoff; Member 2 adds ten room creations/account/hour; Member 4 enforces one active evaluation/session transactionally. Test shared NAT behavior; do not claim the IP limiter implements account controls.

Audit only generated requestId, method, route template, status, stable error category and duration. Do not log bodies, raw URLs/queries, authorization, cookies, passwords/hashes, media tokens, transcript text, model prompts/results or retrieval payloads. Current helpers do not log raw exceptions. Keep diagnostic messages categorical and central collector retention explicit.

Use server-only secret bindings. Never put provider credentials in VITE_ variables, Git, PDF examples or reports. Historical Git removal does not revoke keys. Follow docs/operations.md for generation, network/TLS, backup/restore and incident steps.

8. Required tests and handoff checklist

Before merging a feature, add tests that can fail at its receiving boundary and attach evidence: test ID, build, environment, steps, expected/observed result, outcome and owner. Use synthetic records only. Keep blocked/unrun tests distinct from passes.

Member	Required negative cases before security sign-off
1	Password preserved; generic auth errors; expired/wrong-signed JWT; refresh replay/family revoke; missing/foreign Origin and CSRF; cross-account history
2	Race third join; nonmember/ended-session media; camera/screen denied; replay events; malformed/oversize WS; clock tampering and repeated finish
3	XSS-like plain text; poisoned/cross-account/expired cache; storage disabled; logout/end across tabs; stale offline cache purge before rendering
4	Unknown speaker/score IDs; duplicate segments; oversized buffer; prompt attacks; fabricated citation; private-IP/redirect SSRF; timeout/outage and hard purge
5	Run common negative suite, builds, audits, source secret scans, Compose smoke and synthetic restore; independently review AT-01–10 evidence

Use backend/tests/security.test.ts and supply-chain.test.ts as executable patterns. They verify shared controls and existing routes. They do not substitute for absent account/WS/cache/AI end-to-end tests. The initial handoff records 48 passing tests; refer to report.md for the final count and any later changes.

Release is blocked until feature integrations, historical credential rotation, verified migration startup, ASR/model/provider decisions and complete acceptance evidence are supplied. Public HTTPS, cross-network audio, named-browser accessibility and measured load/latency have not been demonstrated by a local API test.